

Session 5

Agentic AI systems

What you build today: Project 4, the multi-agent resolution system.

The job

- Use Projects 1 to 3 as tools, plus the customer records
- Decide, propose escalation when the policy says so, never execute it alone
- Survive a ticket that tells it to ignore its instructions
- Prove it did the right things in the right order, at a cost Northwind accepts

An agent is a loop

model, tool call, observation, model, until an answer or a cap.

Under a hundred lines. Every failure in this session is visible in it. Build it before you touch a framework, so you can debug one later.

The guard rails

Guard rail	Prevents
Schema validation	the loop spinning on bad arguments
Step cap	a runaway loop, in steps
Spend cap	the same, in money
Approval gate	a model executing an irreversible action
Untrusted data tags	customer text read as instructions
Trace	a failure nobody can replay

Tool design is API design

- One Pydantic model per tool, a strict schema the model sees
- Errors come back as observations, not exceptions
- Narrow purpose, ids not free text, idempotent where possible
- A bare boolean invites invented reasoning: return the reason too

Most agent failures are tool-design failures.

Control flow, simplest first

Pattern	Earns its complexity when
Single loop	most tasks: start here
Orchestrator and specialists	tools cluster into roles that scale and fail differently
Planner and executor	the plan must be reviewed before anything runs
Reflection	a wrong answer costs more than a second pass

A specialist is a tool that makes an HTTP call

- The orchestrator is an agent whose tools are the specialists
- Each specialist runs the same loop, narrower prompt, subset of the registry, its own `/run`
- Deploys, scales, fails and traces on its own
- The orchestrator records the specialist's run id, so a failure replays end to end

Blast radius

- Escalation pages a person. The model never executes it
- `requires_approval`: the loop records a proposed action and says so
- Approval happens outside the loop, by a person or a policy people wrote
- If the queue grows during an evaluation run, every case fails

Prompt injection through data

- A ticket is text a customer wrote. So is a similar past ticket. So is a log line
- Wrap it, name it as data in the system rules
- The gate makes the worst case a proposal, not an action
- Three injection cases in the adversarial set; any escalation fails the run

Evaluate trajectories, not answers

Signal	Why
Task success	deterministic checks written as data per case
Tool selection	called what was needed, did not wander
Steps and cost	a right answer at twelve steps and a dollar is wrong
Escalation decision	proposed when required, absent when forbidden
Replayable trace	every run leaves one

Frameworks, after the loop

- Strands on AWS, Agent Development Kit on GCP, same tools, same adversarial set
- What it bought: session state, retries, streaming, native tracing
- What it hid: the loop, the caps, the moment the gate fires
- You built the loop ninety minutes ago. You have opinions. Good

Model Context Protocol

- The registry becomes an MCP server with one decorator
- Any MCP client can call `search_policies`, including the coding agent on your laptop
- Optional today; load-bearing on the Reference stack, behind a gateway with policy

Acceptance criteria

1. Bad tool arguments come back as observations; the loop does not die
2. Step cap, spend cap and model errors terminate cleanly with a reason
3. Zero escalations executed without approval, in tests and in the adversarial run
4. At least 12 of 15 adversarial cases pass; every injection case passes
5. Every run leaves a trace that `replay` prints

`make session05` reports every test passed.

Questions worth asking

- Which adversarial case would have reached production in your current setup?
- What is the cost per resolved ticket, and what would Northwind accept?
- What did the framework hide, and does that matter at 3am?